

Reviewer Pack — Minimal Rank of Quandles Bounds Resolution Refutation Length

Entry e00c80b4e5a6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 22:31:36 UTC

Minimal Rank of Quandles Bounds Resolution Refutation Length

Entry ID: e00c80b4e5a6 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 22:31:36 UTC

1. Conjecture

Field A (mathematical branch): Quandle Theory **Field B** (complexity object): Complexity Theory: Resolution Proof Complexity

Statement:

{'text': 'For a given Tseitin formula F derived from a graph G , if the minimal rank of the quandle associated with G is $r(G)$, then the minimum resolution refutation length for F is at least $2^{r(G)}$.', 'quantitative_relation': ' $E[X(\text{instance})] = \Theta(2^{(\text{rank_quandle}(G)))$ '}

Rationale (proposer's reasoning):

{'text': 'Quandles are algebraic structures that have not been extensively explored in complexity theory. Their ranks could potentially provide a new lens through which to analyze the hardness of Tseitin formulas, as quandle operations might lead to intricate interactions between clauses.', 'why_structure': 'Quandle operations may capture non-trivial interactions between literals in a formula, potentially leading to exponential lower bounds.'}

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c219e5477b1b9835

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 95% of the random graphs G on n vertices, the minimal rank of the quandle $r(G)$ corresponds to a resolution refutation length of at least $2^{r(G)}$, with an aggregate Spearman's rank correlation coefficient of at least 0.7.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal rank quandle AND resolution proof complexity - Tseitin formula AND quandle theory AND refutation length - Quandle associated with graph AND resolution length lower bound

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=2.8s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_random_graph(n):
        edges = set()
        for i in range(n):
            for j in range(i + 1, n):
                if random.random() < 0.5:
                    edges.add((i, j))
        return edges

    def compute_quandle_rank(graph):
        n = len(graph)
        rank = 0
        while True:
            found_new_element = False
            for i in range(n):
                for j in range(i + 1, n):
                    if (i, j) not in graph and (j, i) not in graph:
                        new_element = set()
                        for k in range(n):
                            if (k, i) in graph and (k, j) not in graph:
                                new_element.add((k, j))
                            elif (k, i) not in graph and (k, j) in graph:
                                new_element.add((k, i))
                        if len(new_element) > 0:
                            found_new_element = True
                            rank += 1
                            graph.update(new_element)
            if not found_new_element:
                return rank
```

```

        break
    return rank

def generate_tseitin_formula(graph):
    n = len(graph)
    literals = {i: f"x{i}" for i in range(n)}
    clauses = []
    for (i, j) in graph:
        clauses.append([literals[i], literals[j]])
        clauses.append([-literals[i], -literals[j]])
        clauses.append([-literals[i], literals[j]])
        clauses.append([literals[i], -literals[j]])
    return clauses

def resolution_refutation_length(clauses):
    stack = []
    while True:
        unit_clause = next((c for c in clauses if len(c) == 1), None)
        if not unit_clause:
            break
        literal = unit_clause[0]
        clauses.remove(unit_clause)
        for clause in clauses[:]:
            if literal in clause:
                clauses.remove(clause)
            elif -literal in clause:
                index = clause.index(-literal)
                clause.pop(index)
                clause.extend([l for l in unit_clause if l != literal])
                stack.append(clause)
    return len(stack)

n = random.randint(5, 40)
graph = generate_random_graph(n)
quandle_rank = compute_quandle_rank(graph)
tseitin_formula = generate_tseitin_formula(graph)
refutation_length = resolution_refutation_length(tseitin_formula)

return {
    "metric_name": "resolution_refutation_length",
    "metric_value": refutation_length,
    "instances_tested": 1,
    "conjecture_holds": refutation_length >= 2 ** quandle_rank,
    "counterexample": f"quandle_rank={quandle_rank}, refutation_length={refutation_length}" if
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

```

```

mean = sum(results) / len(results)
std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
support_fraction = sum(1 for r in results if r >= 2 ** quandle_rank) / len(results)

if support_fraction >= 0.95:
    print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
elif any(r < 2 ** quandle_rank for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if result < 2 ** quandle_rank)
    print(f"RESULT: FALSIFIED counterexample='quandle_rank={quandle_rank}, refutation_length={refutation_length}'")
else:
    print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9bb3020a.py", line 100, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9bb3020a.py", line 83, in run_trial
    tseitin_formula = generate_tseitin_formula(graph)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9bb3020a.py", line 57, in generate_tseitin_formula
    clauses.append([-literals[i], -literals[j]])
                   ^^^^^^^^^^^^^
TypeError: bad operand type for unary -: 'str'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification. | next: Debug the test code to ensure it runs without errors and produces the required data for evaluation.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11263
2	preregistration	ollama_remote	glm4:latest	0	0	5955

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	glm4:latest	0	0	4581
4	novelty	ollama_remote	glm4:latest	0	0	5252
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17206
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20821
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13675
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11344
9	judge	ollama_remote	glm4:latest	0	0	8425

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 98522 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e00c80b4e5a6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e00c80b4e5a6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/e00c80b4e5a6.tar.gz` (if generated)